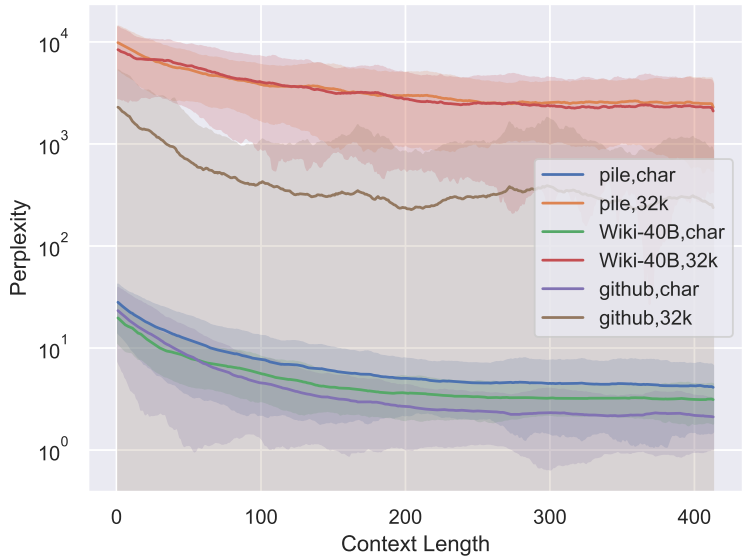

Figure 7: Smoothed Token Perplexity over the Pile, Wiki-40B and Github, with character-level and T5 default vocab

Python:

```
binary_search()z
  if (high >= low)z
    mid = (high + low) // 2
    if (arr[mid] == x)z
      return mid
    if (arr[mid] > x)z
      high = mid - 1
      return binary_search()
    elsez
      low = mid + 1
      return binary_search()
  elsez
      return -1
def func2()z
```

E Semantic Deciphering Full Example

'crash!' 'aaah!' i looked up from my cup of coffee. 'crash!' - that was the cafe window. and 'aaah!' - that was kate. people in the cafe shouted. kate and i ran to the window. there was no one there. then i turned to kate and put my arm around her. 'are you all right?' i asked. 'yes,' she said. 'i think so.' 'what is it?' some one shouted and a short red-faced man ran into the room. the man took my arm. 'matt! what are you doing to kate?' he asked. 'nothing, papa,' kate replied. 'it wasn't him. it was from out in the street.' the red-faced man looked at the window and then at me. he turned to his daughter. 'are you ok, kate?' he asked. kate gave him a little smile. 'yes, i think i am, papa,' she said. then her father spoke to me. 'sorry, matt. i heard kate and i thought...' 'that's ok, paolo,' i answered. it was ok. you see, this is soho, in the centre of london. in the day it's famous for music and films. at night people come and eat and drink

```
in the restaurants.  expensive restaurants and cheap restaurants; italian
restaurants and chinese restaurants.  and day and night there are internet
cafes like the web cafe.  in soho you can buy any thing and any one.  there
are lots of nice people in soho.  but there are also lots of people who are
not very nice.  i know because i live and work here.  i often take a drink to
a shop or cafe.  i'm not rich and famous.  and i don't know a lot.  but i do
know soho.  what one here is a drink - restaurants - music - coffee - father
the one here that drink is
```

Example prediction of the lexinvariant with 32k vocabulary train on the Pile:

```
 - coffee.  and i
```

F Synthetic Reasoning Task

Table 2 shows a variant of the synthetic reasoning task results in Subsection 1, where the symbols are instead sampled proportion to the token frequencies. Although the improvement still generally holds, the standard LM with character-based vocabulary becomes significantly better. We believe that this is because the model can get a significant advantage by guessing among the most common letter.

Dataset	Vocab	LookUp Acc		Permutation Acc	
		Standard	LI	Standard	LI
Pile	char	72.80	90.95	40.63	60.47
	32k	61.20	90.95	40.55	54.55
Wiki-40B	char	75.55	63.45	42.71	59.86
	32k	41.05	57.95	26.81	51.86
Github	char	66.00	86.75	36.62	70.77
	32k	59.25	78.45	37.46	65.04

Table 2: Synthetic Reasoning Tasks (adjusted for token frequencies)

G Language Models Regularized with Lexinvariance and BIG-bench Results

As described in the main paper, we implement a lexinvariance regularized Model in a way similar to embedding dropout. Note that one problem in implementing it naively by using random Gaussian embeddings and learned embedding in a mixture is that the two would become quickly distinguishable from each other during training since learned embeddings often have larger norms, allowing the model simply ignore the randomized tokens. So instead of using random Gaussian embedding matrices in place of a learned embedding matrix, we explored another approach for training a lexinvariant regularized LM: training a standard LM with learnable embedding matrix over sequences partially applied with a random token permutation $B_p(x_1,\pi),\ldots,B_p(x_1,\pi)$, where $B_p(x_i,\pi)=\pi(x_i)$ with probability p and $B_p(x_i,\pi)=x_i$ with probability $1-p$. Since each token can be remapped to any other token with equal chance, the produced model should ideally also be lexinvariant when $p=1$, though with no strict guarantees. In practice, we found the models trained this way behave very similarly to models with random Gaussian embedding.

We evaluate our model over BIG-bench tasks where the language model performance scales well, and we prioritize evaluating generative tasks over multiple-choice tasks. Tasks we evaluated on:

gre reading comprehension.mul, linguistics puzzles.gen, linguistics puzzles.gen, rhyming.gen, tellmewhy.gen, simple arithmetic multiple targets json.gen, simple arithmetic json subtasks.gen, disfl qa.gen, arithmetic.gen, bridging anaphora resolution barqa.gen, matrixshapes.gen, sufficient information.gen, logical args.mul, novel concepts.mul, code line description.mul, unnatural in context learning.gen, unit interpretation.mul, english proverbs.mul, general knowledge.mul, geometric shapes.gen, human organs senses.mul, contextual parametric knowledge conflicts.gen, crass ai.mul, auto categorization.gen, penguins in a table.gen, hindu knowledge.mul, english russian proverbs.mul, modified arithmetic.gen, cryobiology spanish.mul, evaluating information essentiality.mul, intent recognition.mul, understanding fables.mul, figure of speech detection.mul, empirical judgments.mul,

simple ethical questions.mul, swahili english proverbs.mul, language identification.mul, phrase relatedness.mul, nonsense words grammar.mul, undo permutation.mul, object counting.gen, identify odd metaphor.mul, elementary math qa.mul, social iqa.mul, parsinlu qa.mul, metaphor understanding.mul, timedial.mul, causal judgment.mul, list functions.gen, implicatures.mul, date understanding.mul, codenames.gen, fact checker.mul, physics.mul, abstract narrative understanding.mul, emojis emotion prediction.mul, metaphor boolean.mul, strategyqa.gen, ascii word recognition.gen, auto debugging.gen, cause and effect.mul, conlang translation.gen, cryptonite.gen, cs algorithms.mul, dyck languages.mul, gender inclusive sentences german.gen, hindi question answering.gen, international phonetic alphabet transliterate.gen, irony identification.mul, logical fallacy detection.mul, movie dialog same or different.mul, operators.gen, paragraph segmentation.gen, parsinlu reading comprehension.gen, repeat copy logic.gen, rephrase.gen, simple arithmetic json.gen, simple arithmetic multiple targets json.gen, sports understanding.mul, word unscrambling.gen, hyperbaton.mul, linguistic mappings.gen, anachronisms.mul, indic cause and effect.mul, question selection.mul, hinglish toxicity.mul, snarks.mul, vitaminc fact verification.mul, international phonetic alphabet nli.mul, logic grid puzzle.mul, natural instructions.gen, entailed polarity.mul, list functions.gen, conceptual combinations.mul, goal step wikihow.mul, logical deduction.mul, conlang translation.gen, strange stories.mul, odd one out.mul, mult data wrangling.gen, temporal sequences.mul, analytic entailment.mul, disambiguation qa.mul, sentence ambiguity.mul, swedish to german proverbs.mul, logical sequence.mul, chess state tracking.gen, reasoning about colored objects.mul, implicit relations.mul, riddle sense.mul, physical intuition.mul, simple arithmetic json multiple choice.mul, geometric shapes.gen, gem.gen, simp turing concept.gen, common morpheme.mul, qa wikidata.gen, international phonetic alphabet transliterate.gen, similarities abstraction.gen, rephrase.gen, emoji movie.gen, qa wikidata.gen, word sorting.gen, emoji movie.gen, qa wikidata.gen, periodic elements.gen, hindi question answering.gen

Bellow, we plot the net percentage of tasks improved/deproved in each of the BIG-bench categories, out of the tasks that are changed by at least a threshold amount.

H Compute

We use one TPU v3-8 for all our pretraining runs. It takes approximately 23 hours for each pretraining run.

I Broader Impacts

Our work primarily provides a scientific exploration and understanding of the properties of lexinvariant language models. More broadly, these properties could potentially help improve the robustness, generalizability, and reasoning ability of LMs in the future works. In general we don't foresee more specific negative societal impacts from this work other than general misuse of language models.